

WONDER GATHER - HIGH-FIDELITY PROJECT HANDOFF

Prepared for an external architect of AI-assisted project systems

Snapshot date: 23 September 2026

Repository: `MacquePanoramix/Modular-RTS`

Current `main` head:

`d03e0456f0399cd2ad9d1faecfce636f54fe6c47` - *Add
lightweight Wonder Gather continuity layer*

Current gameplay milestone: The Living Body

Milestone lifecycle: Technical Ready - Game

Director playtest pending

Unity: `6000.6.0f1`

This document is a factual handoff, not a proposal
for a new harness.

It describes the project as it actually exists today,
including contradictions, unfinished areas, informal
practices, and recently added continuity machinery.

Evidence-status vocabulary used in this handoff

To avoid blurring fact, interpretation, and design intent, statements below use these meanings:

- **Canonical/current** - explicitly treated by the repository as current project truth.
- **Implemented** - exists in code/assets today; this does **not** automatically mean the underlying design is permanent.
- **Historical** - useful provenance, but superseded as current authority.
- **Informal convention** - real practice, but not necessarily enforced by tooling.
- **Inference** - reconstructed from repository history and the long-running design conversation.
- **Open** - deliberately unresolved.
- **Working-memory context** - known to the high-context design conversation but not fully externalized in durable project artifacts.

1. Project in one page

What the project is trying to build

Wonder Gather is a **slow-paced, fully 3D fantasy RTS in which players design a civilization before the match rather than selecting a predefined faction.**

Its conventional gameplay ancestry is recognizably *Age of Empires*:

starting base → gather → construct → produce →
expand → fight

But its central distinction is that the civilization itself is authored by the player.

A civilization is intended to become a connected network of:

- starting base properties;
- starting units and resources;
- unit blueprints;
- building blueprints;
- construction permissions;
- production relationships;
- resources;
- progression / technology structure;
- body and equipment choices;
- personality and autonomy traits;
- eventually other unusual civilization-specific mechanics.

The project often compares this to **deckbuilding**, except the "deck" is an entire civilization whose possibility graph unfolds during an RTS match.

The second defining pillar is embodiment.

Units are not intended to feel like abstract RTS tokens with animations attached. Their bodies, movement, traits, temperament, courage, and eventually combat should make them worth observing at close range.

The tone explicitly rejects the deliberately slapstick physics of *Totally Accurate Battle Simulator*. TABS is an inspiration for emergence and procedural physicality, but also an anti-reference for tone.

The desired emotional result is grounded, readable, poetic, and quietly wondrous.

Why it exists

The design goal is larger than "make a highly customizable RTS."

The intended pleasure is:

invent a civilization, bring it into being, command it, and enjoy watching its inhabitants physically live out the consequences of what was designed.

The player should not spend the entire match in strategic autopilot staring only at numbers, production queues, and efficiencies.

Close observation is part of the intended play experience.

The project therefore tries to hold two scales at once:

- the civilization / landscape / strategy scale;

- the individual body / gesture / small emergent event scale.

The newer project-culture shorthand for this tension is:

One eye on the constellation, one hand in the soil.

What a successful end state looks like

The long-term intended primary competitive mode is **Open Workshop** multiplayer.

Players would bring previously authored civilizations into matchmaking under shared design-budget and legality rules.

Two civilizations could differ profoundly in:

- starting structure;
- economy;
- progression depth;
- build permissions;
- production topology;
- body types;
- physical capabilities;
- equipment;
- temperament;
- resource usage;
- resilience;
- strategic style.

Despite that freedom, the game must remain:

- legible;
- competitively meaningful;
- physically grounded;
- strategically controllable;
- visually coherent;
- pleasant to observe.

A key locked design distinction is the **two-cost model**:

1. **Design value** - pre-match cost for how much possibility/resilience is accessible from the starting setup.
2. **In-match cost** - resources/time/capacity required to actually produce something during play.

A powerful late unit can have high in-match cost but modest design value if hidden behind a long fragile progression chain. Direct access from the starting base would cost much more design value.

Current stage

This is no longer merely a paper concept.

The repository contains working small-scale versions of:

- 3D RTS camera;
- click / shift-click / box selection;
- group movement;
- NavMesh navigation and local avoidance;
- alternate reachable destination handling;

- finite resource gathering;
- carrying and depositing;
- building placement;
- timed construction;
- unit production queues;
- civilization definitions;
- multiple editable unit blueprints;
- multiple editable building blueprints;
- many-to-many build and production relationships;
- civilization reachability validation;
- player-facing faction creation;
- persistent local faction storage;
- schema migration / corruption safeguards;
- provisional unit-performance customization;
- procedural biped presentation layered over navigation.

The latest documented full Unity PlayMode regression is:

61 passed, 0 failed.

That is evidence for the covered behavior only. It is not evidence that the full game is close to completion.

What is actively being worked on now

The active gameplay milestone is **The Living Body**.

Its question is deliberately narrow:

Can a navigation-driven procedural body make an RTS unit feel grounded, expressive, and worth watching without becoming slapstick or weakening strategic readability?

The current implementation includes:

- world-space support feet;
- alternating steps;
- predictive placement;
- slope-normal adaptation;
- procedural knees;
- variable pelvis height;
- acceleration/body lean;
- arm counter-swing;
- start/stop response;
- redirection behavior.

The gameplay root remains NavMesh-authoritative.

This is not active-ragdoll locomotion.

The current remaining gate is **human experiential acceptance** by the Game Director.

What the next major milestone is

Canonical answer: no next major milestone is currently locked.

`Docs/CURRENT_STATE.md` explicitly says the next action is human playtest first.

After that, the project should either:

1. refine The Living Body because the experiment has not yet answered its question; or
2. deliberately choose the next milestone.

A personality/temperament prototype ("Three Temperaments") is a plausible directional next experiment from the design history, but it is not current canonical project state and should not be treated as scheduled work without the Game Director choosing it.

2. What kind of project this really is

Wonder Gather is best understood as a **creative / technical hybrid containing several research-like problems inside a game-development project**.

It is not merely "an RTS implemented in Unity."

Conventional game engineering

The project includes ordinary software/game systems:

- input;
- selection;
- camera;
- navigation;
- UI;
- persistence;

- resource systems;
- building placement;
- production;
- scene authoring;
- regression tests;
- build tooling.

These areas generally have objective correctness criteria.

Systems-design research

A much harder question is whether an RTS civilization can be represented as a very flexible player-authored dependency graph without becoming:

- incomprehensible;
- impossible to balance;
- trivial to exploit;
- too tedious to author;
- too conventional after restrictions are added.

The project is actively discovering the answer rather than implementing a known design pattern.

Simulation / AI research

The intended future unit model combines ideas that are individually difficult and mutually interacting:

- body/physical traits;
- personality;

- courage and morale;
- autonomy;
- procedural motion;
- combat;
- emergent reactions;
- eventual networking constraints.

The exact behavioral model is open.

Visual / experiential research

Many important success criteria cannot be reduced to tests.

For example:

- does movement feel grounded?
- does a body feel alive rather than robotic?
- is autonomy legible rather than frustrating?
- do close-up events remain interesting after repetition?
- does the game preserve wonder rather than become merely technical?

Words such as "weighty," "floaty," "gentle," "awkward," "alive," and "goofy" are real acceptance vocabulary in this project.

Where uncertainty lives

Highest uncertainty currently exists in:

- final body/customization vocabulary;
- procedural animation depth;

- physics authority;
- combat resolution;
- personality and autonomy;
- morale;
- multiple-resource architecture;
- tech/progression representation;
- design-budget formulas;
- multiplayer authority;
- scale/performance targets;
- final UI;
- art direction;
- individual unit variation;
- long-term persistence compatibility rules.

These areas are intentionally not frozen.

Where correctness matters most

Correctness matters disproportionately in systems where errors can remain silent, persist, or reinterpret player-authored content:

- faction save integrity;
- schema migration;
- stable blueprint IDs;
- asset GUIDs;
- civilization graph semantics;
- payment/refund conservation;
- runtime permissions;
- deterministic interpretation of persistent authored relationships;
- eventual multiplayer authority.

A gait parameter can be thrown away.

A migration that silently changes a player-created civilization is a different class of risk.

3. Where project truth currently lives

The project now has a clearer hierarchy than it did at inception.

Docs/CURRENT_STATE.md

Type of truth: current-state router.

Authority: high for active milestone/lifecycle.

Current: yes.

Can become stale: yes, deliberately; it must be refreshed when the active milestone meaningfully changes.

It answers:

- where are we now?
- what is technically established?
- what remains unproven?
- what is waiting on the Game Director?
- which deeper document should be retrieved next?

Subtlety: it records runtime baseline commit

`d9b5f4d4` because the current docs-only continuity commit did not change gameplay. Actual repository head is now `d03e0456`.

That is not currently treated as a contradiction requiring correction; it records the implementation baseline when the state note was written.

Docs/GAME_VISION.md

Type of truth: canonical living design source of truth.

Authority: highest durable design authority under the Game Director.

Current: yes.

Can become stale: yes if decisions are changed without updating it.

It distinguishes:

- **Locked;**
- **Direction;**
- **Possible;**
- **Open;**
- **Implemented.**

This distinction is one of the most important safeguards in the project.

Implementation does not automatically promote a choice into design law.

Docs/DESIGN_RATIONALE.md

Type of truth: high-value "why" memory.

Authority: supporting rationale, not replacement design authority.

Current: yes, newly added.

Can become stale: less frequently than current state, but yes.

Its purpose is to preserve reasons whose loss could let a competent contributor follow the words of the design while building the wrong experience.

Examples include:

- observation is gameplay;
 - physicality serves embodiment, not physics spectacle;
 - customization should feel like authorship rather than spreadsheet editing;
 - strange civilizations deserve room to breathe;
 - slowness creates attention;
 - implemented is not locked.
-

Docs/PROJECT_CULTURE.md

Type of truth: collaboration vocabulary, context routing, consequence-sensitive working culture.

Authority: mnemonic / operational shorthand only.

Current: yes, newly added.

Can become stale: yes if collaboration patterns change.

It defines optional project-native roles:

- **Keeper of the Horizon** - Game Director;
- **Threadkeeper** - high-context design/continuity AI;
- **Worldsmith** - implementation AI/developer;
- **Guest Cartographer** - fresh independent reviewer for rare high-risk work.

It explicitly states that plain meanings outrank metaphor.

Source code under `Assets/_WonderGather`

Type of truth: what runtime/editor implementation actually does.

Authority: highest implementation truth.

Current: yes.

Can become stale: not in the same sense; code is reality for the checked-out commit, though docs may describe it incorrectly.

Important areas include:

- `Assets/_WonderGather/Scripts/Civilizations/`
- `Assets/_WonderGather/Scripts/Creator/`
- `Assets/_WonderGather/Scripts/Units/`
- `Assets/_WonderGather/Scripts/Commands/`
- `Assets/_WonderGather/Scripts/Selection/`
- `Assets/_WonderGather/Scripts/Camera/`

If prose disagrees with current runtime behavior, inspect code/tests before assuming prose is correct.

`Docs/AI/UnityProjectContext.md`

Type of truth: technical architecture/history and implementation-agent continuity.

Authority: high for engineering context.

Current: maintained through milestones.

Can become stale: yes; later sections supersede earlier constraints.

Its job is to reduce rediscovery across disposable implementation-agent sessions.

Docs/Validation.md

Type of truth: execution evidence, failure history, known limitations.

Authority: high for what was actually tested/run.

Current: updated through Living Body.

Can become stale: yes after new work.

It records:

- focused tests;
- full regression results;
- build results;
- failures;
- corrected assumptions;
- visual-probe findings;
- things explicitly *not* validated.

This document is unusually important because it records failed attempts rather than only final green states.

Milestone playtest documents

Examples:

- **Docs/LivingBodyPlaytest.md**

- Docs/UnitPerformancePlaytest.md
- Docs/BuildingNetworkPlaytest.md
- Docs/FactionLibraryPlaytest.md
- Docs/MultipleBlueprintPlaytest.md

Type of truth: human acceptance questions for a specific milestone.

Authority: high for what the Game Director should manually judge.

Current: milestone-specific.

They distinguish automated correctness from experiential acceptance.

AGENTS.md

Type of truth: repository-level working agreement for agents.

Authority: high for agent behavior.

Current: yes, expanded in d03e0456.

It now instructs agents to:

- start at CURRENT_STATE.md;
- retrieve only relevant deeper context;
- preserve Game Director authority;
- preserve unresolved design questions;
- protect uncommitted human workspace state;
- preserve GUIDs;
- keep runtime free of UnityEditor references;
- validate changes with bounded evidence;
- avoid treating test success as experiential acceptance;
- scale review rigor with consequence.

README .md

Type of truth: repository entry point and current launch instructions.

Authority: medium.

Current: yes.

It now points newcomers toward the current-state/rationale/culture documents.

Automated PlayMode tests

Type of truth: executable invariants.

Authority: high for exactly what each test asserts.

Current: latest documented full suite = 61 green.

They cover behavior across navigation, selection, economy, building, production, faction authoring, persistence, migrations, networks, performance, and procedural locomotion.

They do not prove aesthetics, balance, game feel, or the full intended design.

Git history

Type of truth: provenance.

Authority: high for what changed and when.

Current: yes.

The milestone progression is unusually legible:

Wanderer → Group → Gatherer → Settlement →
Production → Civilization → Faction Creator →
Faction Library → Multiple Blueprints → Building
Network → Unit Performance → Living Body →
Conservatory continuity layer

GitHub issues / pull requests / branches

Current observable state:

- GitHub issues: **none**;
- Pull requests: **one historical merged PR**, the docs-only Conservatory continuity change;
- visible branches: `main` and the continuity branch used for that one-off change.

The project does **not** currently use issues as task truth.

It does **not** currently have a mandatory PR review workflow.

The continuity-layer PR should not be mistaken for a newly established branch policy; it was deliberately used for a self-referential governance/documentation change.

Conversation history / high-context AI memory

Type of truth: rationale, continuity, discarded alternatives, taste, relationships among decisions.

Authority: important but non-canonical.

Current: partially persistent, partially compressed/implicit.

Can become stale/incomplete: yes.

This was once the primary location of the design.

The repository has progressively externalized the most important material because conversation memory should not be the only project archive.

Generated local test reports / builds

Examples:

- local XML test results under ignored `Docs/TestResults/`-style paths;
- local Windows builds such as `Builds/WindowsLivingBody/WonderGather.exe`.

Type of truth: raw local execution artifacts.

Authority: execution evidence when present.

Repository availability: generally not committed.

`Validation.md` preserves the durable summary.

4. Authority and decision-making

Final authority

The human owner / Game Director has final authority over:

- game design;
- creative direction;
- priorities;
- whether an experiment feels right;
- whether an Open/Possible/Direction idea becomes Locked;
- whether a milestone is experientially accepted.

The newer cultural shorthand calls this role **Keeper of the Horizon**.

That title is optional; the authority is not.

What implementation agents may decide autonomously

Implementation AI/developers may normally decide bounded engineering details such as:

- class/component structure;
- local APIs;
- test setup;
- dependency wiring;
- editor authoring code;
- bug fixes needed to meet an explicit milestone;

- implementation-level refactors that preserve semantics.

They should not silently decide unresolved game design.

What requires human approval

Human judgment is especially required for:

- game feel;
 - aesthetic tone;
 - movement quality;
 - camera feel;
 - customization fantasy;
 - autonomy readability;
 - milestone acceptance where the question is experiential;
 - locking previously open design.
-

How architectural decisions are recorded

There is no formal ADR system.

Architecture is currently recorded through a combination of:

- source code;
- `UnityProjectContext.md`;
- `GAME_VISION.md` where architecture intersects design meaning;
- milestone documentation;

- validation history;
 - Git history.
-

Can implementation silently create new semantics?

Formal answer: **it should not.**

The repository repeatedly labels provisional systems as:

- prototype;
- temporary;
- not final;
- open;
- implemented but not locked.

This distinction is now reinforced directly in

`AGENTS.md`, `GAME_VISION.md`, `DESIGN_RATIONALE.md`,
and `PROJECT_CULTURE.md`.

How disagreements are resolved

There is no committee mechanism.

The Game Director has final creative authority.

Technical disagreements are generally resolved through:

- code inspection;
- tests;
- reproduced behavior;
- bounded prototypes;

- occasionally fresh review if risk justifies it.

If the disagreement is about game feel or intended experience, human creative judgment controls.

How a decision becomes official

The clearest durable path is:

1. Game Director decides/accepts a direction;
2. relevant design status in `GAME_VISION.md` is updated;
3. implementation/docs/tests are changed consistently;
4. `CURRENT_STATE.md` changes if milestone state changes.

There is no separate approval database.

5. Current architecture and major conceptual boundaries

RTS command path

Current high-level command architecture:

```
RtsInput
  ↓
SelectionController
  ↓
CommandDispatcher / commands
  ↓
UnitMotor
  ↓
NavMeshAgent
```

`RtsCamera` is separate and consumes input/selection context independently.

This separation was established early and remains important for later autonomy/networking possibilities.

Civilization architecture

Core types include:

- `CivilizationDefinition`
- `UnitBlueprint`
- `BuildingBlueprint`
- `CivilizationValidator`
- `CivilizationSession`

A civilization currently owns a starting setup plus rosters of units/buildings and their relationships.

Civilization graph semantics

A fixed-point reachability model currently determines structural accessibility.

A unit becomes reachable if:

- it starts reachable; or
- a reachable building can produce it.

A building becomes reachable if:

- it is the starting base; or
- a reachable unit can build it.

Cycles are allowed but do not bootstrap themselves.

This graph meaning is foundational enough that later semantic changes would deserve high scrutiny.

Validator boundary

Current validation distinguishes blocking **errors** from non-blocking **warnings**.

It can detect structural issues such as:

- broken IDs;
- duplicate IDs;
- invalid prefabs;
- invalid links;
- unreachable branches;
- absence of reachable gathering capability.

It does not prove:

- competitive balance;
- full economic viability;
- map resource sufficiency;
- military survivability;
- matchmaking legality.

Faction creator boundary

The player-facing creator edits runtime/draft copies rather than mutating canonical authored assets directly.

Current prototype limits include small fixed rosters and a provisional supply economy.

These are scaffolds, not final design limits.

Creator / playtest separation

The creator and playtest are intentionally separated so runtime match state does not silently mutate faction design state.

The creator remains loaded while a playtest scene can be loaded additively; returning removes the runtime playtest state.

Persistence boundary

Current faction schema is **version 4**.

Important properties include:

- explicit DTO records;
- stable IDs;
- schema versioning;
- unknown-field rejection;
- migrations from older versions;
- migrations occur in memory;

- old data is rewritten only after explicit save/rename;
- optimistic hash/state tokens;
- temp-write + atomic replacement;
- `.bak` previous-version backup;
- deletion recovery directory.

Save files represent civilization designs, not ongoing matches.

Unit-performance boundary

Current performance values are:

- movement percentage;
- carry capacity;
- gathering percentage;
- construction percentage.

They are intentionally described as **temporary outcome controls**.

They should not be mistaken for the intended final fantasy-facing customization language.

Living Body boundary

Current procedural movement architecture is deliberately hybrid:

- NavMesh owns logical/root movement;
- procedural presentation reacts to measured root motion;
- feet/legs/body adapt visually;

- physics does not currently determine gameplay position.

This preserves RTS control while testing whether more embodiment is valuable.

Important invariants / boundaries

Current project architecture strongly favors:

- explicit dependencies;
 - small components;
 - stable GUID preservation;
 - runtime/editor assembly separation;
 - no `UnityEditor` references in runtime;
 - non-destructive scene generation;
 - retaining old milestone scenes for regression/reference;
 - validating candidate group destinations before issuing movement;
 - preserving previous valid movement when a new order is invalid;
 - remembering original production payment/duration for refund/cancellation semantics;
 - protecting human uncommitted workspace state.
-

Intentionally deferred major systems

- combat;
- personality;
- courage/morale;

- arbitrary body anatomy;
 - active ragdolls;
 - multiple final resources;
 - full custom progression;
 - design-budget pricing;
 - custom base property system;
 - multiplayer;
 - DOTS/ECS adoption;
 - final art/UI;
 - localization;
 - controller support;
 - large-scale performance architecture.
-
-

6. Current workflow

The observed workflow is roughly:

```
idea / human reaction
    ↓
clarify the question
    ↓
small bounded milestone
    ↓
implementation in an isolated/safe working context
    ↓
focused automated tests
    ↓
rendered/runtime visual probe when relevant
    ↓
correction
    ↓
full regression run
    ↓
Windows development build
    ↓
validation/playtest documentation
    ↓
commit
    ↓
human experiential playtest
    ↓
next question
```

Idea / milestone definition

Usually begins in conversation with the Game Director.

The important discipline is preserving open design questions instead of opportunistically deciding them because code requires a temporary value.

Artifact:

- milestone request / design discussion;
- sometimes update to `GAME_VISION.md`.

Common failure risk:

- implementation convenience becoming accidental design authority.
-

Implementation

Historically much work has been performed in an isolated validation/development copy rather than directly over the user's active working directory.

This practice arose partly to preserve local human modifications.

Artifacts:

- C#;
- scenes/prefabs/materials;
- editor authoring utilities;
- tests.

Common failure risk:

- misunderstood scope;
 - test setup assumptions;
 - provisional architecture becoming over-generalized.
-

Scene/prefab authoring

The project prefers Unity editor authoring APIs.

Setup scripts often create new experimental scenes and refuse destructive overwrite.

This has produced preserved vertical slices rather than one continuously mutated scene.

Focused testing

New behavior gets narrow tests first.

Failures are investigated rather than simply weakening assertions.

The validation log contains multiple examples where the test itself was wrong, and the project records that distinction.

Visual/runtime probe

For UI/procedural movement, scripted captures have been used to inspect:

- clipping;
- layout;
- body pose;
- slope contact;
- selection-ring orientation;
- scroll states.

Temporary capture code is removed before final build.

Regression

After focused behavior stabilizes, the broader PlayMode suite is run.

Latest documented full suite:

61 passed, 0 failed.

Build

Milestones commonly produce a Windows x64 development build.

The implementation agent does not necessarily perform a full human-feel play session of that standalone build.

Human acceptance

Playtest documents tell the Game Director what to inspect.

This is where work often reaches a genuine integration bottleneck: technical work may be ready, but the next direction depends on high-context human judgment.

7. AI and agent usage

High-context conversational/design AI - Threadkeeper

Effective responsibilities:

- retain/reconstruct design history;
- interpret why systems exist;

- compare current implementation with original intent;
- help shape milestones;
- detect semantic drift;
- distinguish scaffold from intended end state;
- integrate information across many project surfaces.

It should not be treated as the authoritative store of current code state.

The repository now explicitly tries to move retrievable state out of this role so attention can be spent on integration rather than storage.

Implementation AI/developer - Worldsmith

Repository history describes Codex/AI-assisted implementation handling substantial Unity engineering:

- code;
- scene authoring;
- editor utilities;
- tests;
- debugging;
- build/validation work;
- documentation updates.

These sessions appear task/milestone scoped rather than permanently shared-memory agents.

Continuity is reconstructed from repository artifacts.

Game Director - Keeper of the Horizon

Human role.

Controls:

- creative authority;
- milestone priority;
- experiential acceptance;
- design locking/unlocking;
- interpretation of whether a technically valid outcome belongs in Wonder Gather.

This role is intentionally not automated away.

Fresh independent reviewer - Guest Cartographer

This is newly formalized as **optional**, not routine.

Use case:

- silent + consequential risk;
- e.g. save migrations, stable IDs/GUID semantics, graph-meaning rewrites, destructive data changes, future multiplayer authority.

There is no standing independent-review process today.

Persistent versus disposable context

- Human Game Director: persistent.

- High-context conversational AI: relatively persistent but non-canonical and imperfect.
 - Implementation AI sessions: comparatively disposable/task scoped.
 - Repository: durable shared memory.
-

Do agents share context directly?

No evidence of a live shared multi-agent memory fabric.

Context transfer happens via:

- Git repository;
 - current-state/design/architecture docs;
 - tests;
 - validation history;
 - the human owner;
 - prompts/handoffs.
-

Do agents mutate the repository?

Yes, implementation agents can directly implement and author project changes.

The high-context design role is also capable of repository documentation updates when explicitly asked, but should not silently mutate gameplay/design state.

Common AI failure modes observed

Observed failures include:

- wrong/incomplete API calls causing compile failure;
- test setup omitting necessary state;
- edge-condition ordering errors;
- incorrect assertion overloads;
- UI sizing assumptions;
- technically passing behavior that still violates a stronger physical invariant;
- initial inability to reconstruct all design context because key rationale lived only in conversation.

The project's response has generally been stronger evidence/documentation, not broad bureaucracy.

8. Failure history and scar tissue

This section is intentionally concrete.

Failure A - Design context was not durable enough

What happened

The project began primarily as a long design conversation. When implementation started, an earlier detailed design dossier was referenced but not available to the implementation agent.

Why

Conversation context had been functioning as the effective specification.

How detected

Repository bootstrapping revealed that the agent could not independently recover all intended design truth.

What changed

`Docs/DesignBrief.md` was created from recovered context; later `Docs/GAME_VISION.md` became the canonical living design source.

Scar tissue

The project now explicitly distinguishes design statuses and has recently added `CURRENT_STATE.md` and `DESIGN_RATIONALE.md` to reduce dependence on conversational memory.

Failure B - Unity/package incompatibility

What happened

The initial template/package state included an Input System version incompatible with removed APIs.

Why

Editor/package version mismatch.

Detected by

Compilation/import.

What changed

Dependency versions were updated/pinned for the current Unity 6.6 environment.

Scar tissue

Package lock state and Unity version are treated as concrete technical context.

Failure C - Group arrival crowding

What happened

First group movement spacing caused crowding near destinations.

Why

1.8-unit spacing was insufficient for arriving agents among already settled neighbors.

Detected by

Runtime testing.

What changed

Prototype spacing increased to 2.4.

Scar tissue

Group movement validates separate reachable destinations rather than issuing every unit the same point.

Failure D - residual movement on lifecycle disable

What happened

A disabled gathering unit could retain residual motion after path reset.

Why

`ResetPath` alone was insufficient to guarantee a stopped agent state.

Detected by

A stricter lifecycle assertion.

What changed

Agent stop state and velocity clearing became explicit.

Scar tissue

Tests increasingly check invariants beyond broad end results.

Failure E - tests themselves were sometimes wrong

Examples include:

- construction test assuming the wrong world location;
- unit-performance test omitting `ChooseBuilding` before placement;
- gathering deadline assuming too many trips in too little time;
- wrong NUnit overload in building-network assertions.

Why

The test harness encoded assumptions that were not actually part of runtime behavior.

Detected by

Failed focused runs and manual inspection.

What changed

Tests were corrected without weakening runtime expectations.

Scar tissue

`Validation.md` carefully distinguishes runtime failure from test/setup failure.

Failure F - persistence error filter omitted a real category

What happened

Initial faction persistence run passed only 5/8 tests.

Why

`InvalidDataException` was missing from the storage-error handling filter.

Detected by

Persistence tests.

What changed

Error handling was corrected.

Later persistence work also added:

- save locks;
- hash/state conflict detection;
- backups;
- recovery behavior.

Scar tissue

Persistence is treated more defensively because player-authored civilizations are high-value state.

Failure G - validation-order edge case in multiple blueprints

What happened

`AddUnit` returned early at roster capacity before checking whether a supplied duplicate blueprint belonged to the faction.

Why

Validation checks were ordered incorrectly.

Detected by

Boundary testing.

What changed

Ownership validation moved ahead of the limit return.

Failure H - UI problems visible only in rendered output

Observed examples:

- incorrectly scaled arrows;
- clipped note text;
- horizontal overflow;
- clipped footer;
- long-name overflow.

Why

Logic tests did not encode layout/readability.

Detected by

Rendered screenshot probes.

What changed

Iterative layout corrections followed by repeated probes.

Scar tissue

Green logic tests are not treated as UI acceptance.

Failure I - Living Body hid physical inconsistencies

What happened

The Living Body work exposed several different categories of failure:

- compile error from a missing raycast argument;
- constant-leg-length assertion found an approximately 0.688m leg against 0.68m intent;
- visual inspection revealed excessive crouch and feet landing behind the root;
- selection rings remained horizontal and intersected slopes.

Why

Different parts of the implementation violated different invariants: API correctness, geometric constraint, visual plausibility, presentation alignment.

Detected by

- compile;
- stronger automated invariant;
- rendered screenshots.

What changed

- API corrected;
- pelvis minimum-height clamp removed;
- rest posture and predictive landing placement revised;
- arm counter-swing tied to actual steps;
- selection rings aligned to support normals.

Scar tissue

Procedural movement now demonstrates especially clearly that tests and visual judgment are complementary evidence systems.

Failure J - high-context AI was becoming a coordination bottleneck

What happened

An external architecture questionnaire asked the high-context AI to reconstruct nearly the entire project: design, truth surfaces, architecture, history, failures, governance, AI roles, continuity risk, and its own role.

Why

Although project state had become more durable, cross-project meaning still depended heavily on one context-rich conversational role.

Detected by

The handoff exercise itself.

What changed

The project adopted a lightweight continuity layer:

- `CURRENT_STATE.md`;
- `DESIGN_RATIONALE.md`;
- `PROJECT_CULTURE.md`;
- expanded `AGENTS.md` context routing;
- explicit human-workspace protection.

Scar tissue

New shorthand:

durable state belongs in retrievable artifacts;
high-context intelligence should spend attention
on integration and meaning.

9. Existing harness / governance / process machinery

Wonder Gather already has a functioning lightweight harness.

`AGENTS.md`

Why it exists: fast behavioral constraints for agents.

Actually used: yes; repository structure and recent work align with it.

Overhead: low.

Failures mitigated: context overloading, design-authority drift, workspace destruction, evidence overclaiming.

CURRENT_STATE.md

Why: eliminate expensive "where are we?" reconstruction.

Actually used: newly added; too early for long-term evidence of effectiveness.

Overhead: intentionally tiny.

Risk: can become stale if milestone changes are not reflected.

GAME_VISION.md

Why: durable design authority and status separation.

Actually used: yes; maintained over project evolution.

Overhead: moderate but high value.

Failures mitigated: implementation accidentally becoming design law; lost design context.

DESIGN_RATIONALE.md

Why: preserve a small set of dangerous-to-lose "whys."

Actually used: newly added.

Overhead: intended to remain low by limiting

entries.

Failure targeted: semantic drift that still looks superficially compliant.

PROJECT_CULTURE.md

Why: define collaboration roles, context routing, workspace protection, and consequence-sensitive review in memorable language.

Actually used: newly added.

Overhead: low if treated as optional shorthand rather than bureaucracy.

UnityProjectContext.md

Why: disposable implementation-agent continuity.

Actually used: yes, updated milestone by milestone.

Overhead: moderate.

Failures mitigated: rediscovery and violation of earlier architectural boundaries.

Validation.md

Why: preserve execution history, failures, tests, builds, and limitations.

Actually used: extensively.

Overhead: meaningful but directly tied to milestones.

Failures mitigated: false confidence and repeated mistakes.

Milestone playtest docs

Why: separate objective verification from Game Director experiential judgment.

Actually used: yes.

Overhead: low/moderate and task-specific.

PlayMode regression suite

Why: protect accumulated behavior while rapidly iterating with AI.

Actually used: constantly.

Growth: from a few early tests to 61 current regression tests.

Failures mitigated: regressions across movement/economy/creator/persistence/locomotion.

Editor scene-authoring utilities

Why: deterministic/non-destructive scene creation.

Actually used: yes.

Failure mitigated: accidental overwrites and fragile manual scene setup.

Stable-ID + schema/migration machinery

Why: preserve player-authored faction identity across versions.

Actually used: yes.

Overhead: relatively high, justified by consequence.

Atomic save/recovery machinery

Includes:

- temporary writes;
- replacement;
- `.bak` files;
- locks;
- hash conflict detection;
- deleted-item recovery.

Why: protect player-authored civilizations.

Git workflow

Current reality:

- mostly `main`-centric history;
- no active issue tracker;
- one historical PR used specifically for the docs-only continuity/governance change;
- no evidence that mandatory PRs are normal project policy;
- no visible repository CI gate currently serving as authoritative validation.

Validation is primarily local + documented.

10. Coordination bottlenecks

Primary scarce resource: creative integration

The main bottleneck is not writing code.

The scarce resource is answering:

Does this technically plausible thing still belong to Wonder Gather?

That judgment currently belongs to the Game Director.

It is especially required for:

- movement;
- pacing;
- aesthetic tone;
- customization meaning;
- autonomy;
- UI philosophy;
- choosing the next experiment.

High-context integration

The Threadkeeper role is useful because the project contains relationships among decisions made far apart in time.

Example:

"procedural bodies" cannot be evaluated purely as a locomotion feature because they connect to:

- observation-as-gameplay;
- unit attachment;
- future personality;
- combat;
- performance scale;
- multiplayer authority.

The continuity layer reduces raw reconstruction cost, but cross-system meaning still requires integration.

Manual playtesting

A number of milestones become technically ready before they become experientially accepted.

The current project is exactly in that state.

That makes the Game Director a legitimate waiting point, not an avoidable inefficiency.

Context transfer to new agents

This was previously expensive.

It should now be cheaper because:

- agents begin at `CURRENT_STATE.md`;
- then follow only the relevant context path.

This improvement is new and not yet validated over many future milestones.

Implementation versus integration bottlenecks

Implementation bottleneck: comparatively low at present. AI-assisted engineering has produced small slices quickly.

Integration bottleneck: high and increasing as systems become more interdependent.

Questions such as "what should become generic now?" or "is this behavior desirable enough to justify architectural cost?" increasingly dominate over raw coding difficulty.

Knowledge dangerous to lose

Especially dangerous:

- civilization-as-deck/network;
- design value vs in-match cost;
- customization as meaningful fictional authorship, not raw sliders;
- procedural physicality as embodiment, not slapstick spectacle;
- simple commands + rich readable interpretation;
- observation as part of play;
- warnings preferred over prohibition for strange designs where possible;
- progression itself is player-authored and may be absent;
- implementation is not design authority.

Most of this is now durable in `GAME_VISION.md` and `DESIGN_RATIONALE.md`.

11. Reversibility and risk map

Cheap and reversible

Examples:

- placeholder colors/materials;
- HUD copy;
- movement tuning values;
- temporary unit costs;
- test-scene geometry;
- screenshot probes;
- provisional gait timing.

Failure cost: low.

Appropriate process: lightweight iteration.

Expensive but reversible

Examples:

- creator UI architecture;
- locomotion algorithm;
- provisional performance model;
- economy behavior;
- faction-editor interaction flow.

Failure cost: moderate; redesign/refactor possible.

Appropriate process: focused tests + regressions + human inspection.

Difficult to reverse

Examples:

- save-schema semantics;
- stable blueprint IDs;
- civilization graph meaning;
- command/interface contracts after many systems depend on them;
- player-data compatibility promises.

Failure cost: high.

Appropriate process: explicit design reconciliation, stronger evidence, possibly fresh review.

Potentially project-corrupting

Examples:

- silent destructive save migration;
- broken GUID identity;
- mass rewriting of player-created factions;
- implementation silently converting open design into durable semantics;
- future multiplayer authority architecture that conflicts with physical simulation assumptions.

Failure cost: very high and potentially silent.

Appropriate process: isolate candidate, scrutinize architecture, fresh independent review before trust.

Errors easy to detect

- compile errors;
 - broken references;
 - failing deterministic tests;
 - invalid JSON/schema;
 - obvious UI clipping;
 - unavailable NavMesh routes.
-

Errors likely to remain silent

- migration that preserves syntax but changes meaning;
 - semantic drift from original creative intent;
 - stale project-state docs;
 - accidental reliance on provisional assumptions;
 - save compatibility differences older builds cannot understand;
 - physical behavior that passes invariants but feels wrong.
-

Errors likely to compound

- worker-specific assumptions leaking into future generic unit architecture;
- single-resource assumptions becoming structural;
- persistence DTOs becoming coupled to temporary prototype semantics;

- final customization model being built around raw outcome sliders;
- networking introduced after gameplay assumes unconstrained local-only physical authority.

These are risk directions, not claims that all have already happened.

12. What should probably NOT be formalized

Several areas would currently be harmed by more governance.

Exact procedural movement style

Still exploratory.

Do not create rigid gait policy.

Final trait vocabulary

Strength, dexterity, courage, discipline, etc. are not a finished schema.

Do not freeze a taxonomy prematurely.

Resource system

Current **supplies** is a proving scaffold.

Do not create governance around it as though it were the final economy.

Design-budget formula

The two-cost concept is locked; exact mathematics are not.

Do not formalize formulas before enough prototypes exist.

Tech/progression representation

Player-designed progression is core, but the concrete representation remains open.

Do not standardize a conventional tech tree just because it is familiar.

Combat physics

Active ragdolls, hit resolution, weapon contact, balance, and damage semantics are unresolved research areas.

Do not govern implementation that has not yet earned a stable shape.

Low-risk tuning

Camera speed, foot lift, temporary cost values, test colors, wording, and placeholder proportions do not need review ceremony.

Mandatory PR / CI / ADR bureaucracy

This does not currently need governance.

There is no demonstrated coordination problem that justifies mandatory PR review, broad ADR production, or remote CI as a prerequisite to ordinary exploration.

That may change if team size, external releases, or compatibility obligations change.

13. Continuity and succession risk

If the implementation AI disappeared tomorrow

Loss would be manageable.

A successor could reconstruct most current technical state from:

- source code;
- `CURRENT_STATE.md`;
- `UnityProjectContext.md`;
- `Validation.md`;
- playtest docs;
- Git history;
- tests.

This is a major improvement over project inception.

If the high-context Threadkeeper disappeared tomorrow

More nuance would be lost, but much less than before the continuity work.

Harder-to-reconstruct areas would include:

- relationships among distant design discussions;
- intensity of creative preferences;
- discarded alternatives;
- subtle meanings of "wonder," "gentle commander," "grounded," and "alive";
- why a particular compromise felt dangerous even if it was technically valid.

`DESIGN_RATIONALE.md` now protects some of this.

If the Game Director disappeared tomorrow

This is the most severe continuity loss.

The repository can preserve decisions and reasoning.

It cannot authoritatively decide whether:

- movement feels alive;
- UI feels inviting;
- a compromise belongs aesthetically;
- the game still expresses the intended emotional center.

The project intentionally preserves this human authority rather than pretending to eliminate it.

What can be reconstructed from the repository alone

A successor can determine with good confidence:

- active gameplay milestone;
 - current runtime architecture;
 - save format;
 - test state;
 - most locked design pillars;
 - historical milestone sequence;
 - current known limitations;
 - agent working constraints;
 - recent continuity/governance additions.
-

What remains difficult to reconstruct

- uncommitted local Unity state;
 - current human judgment that has not yet been written down;
 - design rationale not yet promoted into `DESIGN_RATIONALE.md`;
 - recent conversational alternatives;
 - exactly what the Game Director currently wants to prioritize if not written in `CURRENT_STATE.md`.
-

Can a successor determine current project state from the repository alone?

Yes, substantially.

That is now one of the project's strengths.

The new continuity layer was specifically added to improve this.

14. Current unresolved tensions

Do not resolve these in a harness.

They are game-design/architecture tensions that require future experiments.

Freeform civilization design vs multiplayer balance

The project wants enormous authorship freedom and fair matchmaking.

Both are central.

Fictional customization vs direct numeric control

Prototype percentages are useful for proving effects.

The final creative language should ideally be body/equipment/personality/structure choices that generate many outcomes indirectly.

The mapping is open.

Physical emergence vs RTS responsiveness

More physical authority may create richer individuality.

It may also harm control, performance, determinism, and networking.

Unit autonomy vs player agency

Personality should matter enough to affect behavior.

Too much disobedience makes the game feel broken.

Readable causality is likely essential, but exact boundaries are unresolved.

Individual variation vs competitive predictability

Variation could create attachment and stories.

It could also make identical blueprints strategically inconsistent.

Grandeur vs intimacy

The world should feel large while battles remain readable enough to appreciate individual bodies.

Actual scale targets are open.

Open Workshop freedom vs exploitability

Bringing saved custom civilizations into matchmaking creates enormous future legality/pricing challenges.

Warnings vs hard restrictions

The Game Director prefers warnings for strange or fragile civilizations.

Some multiplayer-invalid constructions will eventually require actual rejection.

That line is unresolved.

Player-authored progression vs standardized balancing assumptions

A faction can have deep progression, shallow progression, or none.

How design value fairly compares those structures is unresolved.

Current worker lineage vs generic capability architecture

The prototype historically grew from a worker-centric model.

The intended final unit model is much broader.

When to generalize without prematurely rewriting everything is open.

NavMesh-root embodiment vs future physical authority

The current Living Body cleanly separates gameplay root and presentation.

Whether later bodies gain partial/full physical authority remains open.

15. Three representative work episodes

Episode 1 - smooth extension: worker production

Request

After construction was accepted, add unit production.

Path

- production definition added;

- queue/payment/refund semantics implemented;
- safe spawn placement added;
- produced units registered into RTS systems;
- HUD controls added;
- focused tests run;
- full regression suite run;
- visual/runtime probe checked;
- Windows build produced;
- validation/playtest docs updated.

Outcome

22 gameplay tests passed, 0 failed at that milestone.

What it reveals

When architecture and milestone scope are clear, AI-assisted implementation progresses quickly and coherently.

Episode 2 - serious coordination problem: reconstructing project truth

Request

Begin implementing the large game concept from prior discussion.

Problem

The implementation agent did not actually have the detailed design context that the project implicitly assumed existed.

Path

- absence of durable design truth discovered;

- conversation history mined for confirmed decisions;
- initial `DesignBrief.md` produced;
- later replaced/superseded by the more complete `GAME_VISION.md`;
- project gradually externalized architecture/history/validation as well.

Outcome

The repository became capable of carrying far more project continuity independently of one chat.

What it reveals

The project's most important process machinery arose from real continuity pain, not from abstract governance ambition.

Episode 3 - representative current work: The Living Body

Request

Begin testing whether units can feel physically alive through procedural locomotion.

Path

- isolated new scene created;
- shared rig/materials/ramp/NavMesh authored;
- compile issue fixed;
- focused tests passed;
- stronger constant-leg-length invariant added;
- that invariant exposed a hidden geometric error;
- rendered captures exposed crouch/foot-placement issues;

- those were corrected;
- later capture exposed slope-intersecting selection rings;
- ring orientation corrected;
- temporary probe removed;
- full suite run: 61/61 passed;
- Windows build produced;
- human playtest guide written.

Current outcome

Technical Ready - Game Director playtest pending.

What it reveals

This is the mature Wonder Gather loop: small experiment, multiple evidence modes, explicit limits, then human integration.

16. What the high-context AI currently does for the project

The effective role is now called **Threadkeeper** in project culture.

That name is mnemonic, not authority.

What people rely on this role to remember/reconstruct

- original game concept;
- why particular mechanics were proposed;
- how current scaffolds relate to intended systems;

- creative priorities;
 - earlier alternatives;
 - how apparently separate systems connect;
 - the emotional purpose behind technical choices.
-

Judgments this role routinely makes

- core design vs incidental prototype;
 - whether a question needs a decision now or should remain open;
 - whether a proposed technical solution preserves the intended experience;
 - whether the project is drifting toward conventional RTS abstractions;
 - which context actually matters for a new task;
 - how to translate vague human reactions into a bounded next experiment.
-

Context this role historically carried that was poorly externalized

The weakest externalization was often not the rule but the **reason**.

Example:

"procedural bodies are important" is weaker than:

watching a civilization you authored physically exist is itself part of the reward.

The new `DESIGN_RATIONALE.md` exists specifically to reduce this vulnerability.

Mistakes this role is particularly useful at preventing

Semantic substitutions such as:

- "custom units" → raw stat sliders;
- "procedural bodies" → physics for its own sake;
- "player-designed progression" → conventional fixed tech tree;
- "wonder" → fantasy visual dressing;
- "autonomy" → random order refusal;
- "slow" → merely making timers longer.

These mistakes can satisfy a shallow reading of requirements while damaging the intended game.

Where this role was becoming overloaded

It was previously asked to function simultaneously as:

- design historian;
- project-state database;
- architecture index;
- rationale archive;
- milestone planner;
- integration reviewer.

That is exactly what the recent external-handoff exercise exposed.

The project's response was not "give the AI even more context."

It was to improve retrieval and durable memory so high-context attention can be spent on integration.

What parts should live durably instead

Already moved or moving into the repository:

- active state → `CURRENT_STATE.md`;
- locked/open design → `GAME_VISION.md`;
- high-value reasons → `DESIGN_RATIONALE.md`;
- technical history → `UnityProjectContext.md`;
- execution evidence → `Validation.md`;
- working-agent constraints → `AGENTS.md`;
- collaboration/risk shorthand → `PROJECT_CULTURE.md`.

The Threadkeeper should not become the filesystem.

17. Do not misunderstand these things

1. The current worker-derived unit model is not the final unit architecture.

It is historical scaffolding from the earliest RTS loop.

2. `supplies` is not the final economy.

It is one provisional resource used to prove gathering/building/production.

3. Unit-performance percentages are not the intended final customization fantasy.

They are temporary outcome controls.

4. The current faction creator is not the final creation UI.

It is a functional relationship editor/proving environment.

5. Schema/version identifiers do not define the final game's conceptual vocabulary.

They describe the current persistence implementation.

6. The Living Body is not active-ragdoll locomotion.

NavMesh still owns gameplay movement.

7. Reachability is not balance.

The validator answers a structural graph question only.

8. Sixty-one passing tests do not mean the game is complete or the milestone is experientially accepted.

They prove covered technical invariants.

9. The multiplayer-focused game has no multiplayer implementation yet.

This is deliberate sequencing.

10. Implemented is not Locked.

This distinction is foundational.

11. The new Conservatory vocabulary is mnemonic, not a layer of authority above plain engineering terms.

If metaphor creates confusion, discard the metaphor.

12. The project does not currently have a mandatory PR/CI/reviewer bureaucracy.

The one merged PR was a deliberate docs-only continuity-layer change, not evidence of a standing policy.

13. `CURRENT_STATE.md` being technically oriented around `d9b5f4d4` while repo head is `d03e0456` is not gameplay divergence.

`d03e0456` changed continuity/docs only.

14. The apparent simplicity of the current prototype is deliberate.

The project is validating one difficult axis at a time.

18. Artifact map

Artifact	Purpose	Authority	Current?	Why it matters
<code>Docs / CURRENT_STATE.md</code>	Current milestone router	High for present state	Yes	Fast answer to "where are we now?"
<code>Docs / GAME_VISION.md</code>	Living design source of truth	Highest durable design	Yes	Separates Locked / Direction / Possible / Open /

Artifact	Purpose	Authority	Current?	Why it matters
				Implemented
Docs/ DESIGN_RATIONALALE.md	Preserve dangerous-to-lose "why"	Supporting design rationale	Yes	Prevents semantic drift that still looks compliant
Docs/ PROJECT_CULTURE.md	Roles, context routing, risk-sensitive working culture	Operational shorthand	Yes	Defines lightweight collaboration without bureaucracy
AGENTS.md	Agent working agreement	High operational authority	Yes	Tells agents what to read, protect, and not overclaim
Docs/AI/ UnityProjectContext.md	Technical continuity/history	High technical context	Yes	Allows disposable implementation agents to reconstruct architecture
Docs/ Validation.md	Tests, failures, builds, limitations	High execution evidence	Yes	Best source for real scar tissue
Docs/ LivingBodyPlaytest.md	Current human acceptance guide	High for current experiential gate	Yes	Defines what remains for the Game

Artifact	Purpose	Authority	Current?	Why it matters
				Director to judge
Assets/ _WonderGather/ Scripts/ Civilizations/	Civilization runtime/graph model	Highest implementation truth	Yes	Core blueprint/reachability semantics
Assets/ _WonderGather/ Scripts/ Creator/	Player faction authoring/persistence	Highest implementation truth	Yes	Current editor/save architecture
Assets/ _WonderGather/ Scripts/ Units/ Procedural Biped.cs	Current body experiment	Highest implementation truth	Yes	Defines what Living Body actually is today
Git main history	Provenance	High	Yes	Shows how the project actually evolved

Historical but useful:

- Docs/DesignBrief.md - early recovered design context; superseded by GAME_VISION.md.

19. Confidence and missing evidence

High confidence

This handoff is high-confidence about:

- current repo head;
- current milestone;
- Unity version;
- current test/build status as documented;
- civilization graph semantics;
- current faction save/version architecture;
- present role/authority split;
- current continuity-layer documents;
- historical milestone sequence;
- major documented failures;
- areas explicitly not implemented;
- lack of active GitHub issues;
- existence of one merged continuity-layer PR.

These points were checked against the current repository rather than recalled only from conversation.

Moderate confidence / inference

These depend partly on observed practice rather than a formal enforcement mechanism:

- how consistently future implementation agents will begin at `CURRENT_STATE.md`;

- whether the new Conservatory role vocabulary will naturally persist;
 - how often future work will use branches/PRs;
 - whether every implementation session uses the same AI configuration;
 - whether all local work always happens in isolated copies before final commit;
 - whether there are informal task lists outside the repository.
-

Important materials not inspected / not available

This handoff does not include direct inspection of:

- any uncommitted local Unity working tree state;
 - ignored raw XML test reports;
 - local Windows build binary contents;
 - private agent transcripts;
 - private collaborator notes;
 - any external task tracker not surfaced in the repository;
 - future design discussion that has not yet been promoted into durable files.
-

Clarifications an external architect should ask the Game Director if needed

If the later harness-design question depends on them, ask:

1. Has the Living Body been manually playtested since `CURRENT_STATE.md` was written?
 2. Has a next milestone already been chosen outside the repository?
 3. Is there important uncommitted local Unity work not represented on GitHub?
 4. Which implementation AI/environment is currently being used for the next coding milestone?
 5. Does the Game Director want the current Conservatory vocabulary to remain playful shorthand only, or become a long-term part of the project's identity?
 6. Has any external collaborator begun depending on save compatibility or repository workflow promises?
 7. Is any near-term multiplayer work planned, or is it still intentionally deferred?
-

Bottom-line characterization for the external architect

Wonder Gather is a **young, highly exploratory, AI-assisted game project whose technical execution is moving faster than its creative/architectural questions can safely be frozen.**

Its most important coordination problem is not lack of coding capacity.

It is preserving coherence while a very flexible game concept becomes concrete through experiments.

The project already has a lightweight but meaningful harness:

- canonical design truth;
- a current-state router;
- high-value rationale memory;
- technical continuity;
- durable validation history;
- automated regression coverage;
- human experiential acceptance;
- explicit agent constraints;
- consequence-sensitive review guidance.

The project has deliberately **not** adopted heavy governance where no real problem yet justifies it.

The human Game Director remains the irreplaceable creative integrator.

The implementation AI is productive when tasks are bounded.

The high-context AI is most valuable as an interpreter of relationships and rationale, not as the sole archive.

The repository is now increasingly capable of transferring technical state between agents on its own.

The most important remaining continuity risk is not "where is the code?"

It is preserving enough **why** that future contributors can build the right game rather than merely satisfy the right nouns.

That is the project as it actually exists today.